Workshop Banner

Open in Colab

Session 02 // Module 01: Retrieval-Augmented Generation (RAG) for Petroleum Geoscience

This module builds a practical RAG system over a curated geoscience dataset. You'll ingest domain data, chunk and embed it, store it in a vector database, retrieve relevant context, and generate concise, cited answers with a local LLM.

Learning Objectives

- Understand RAG components: splitter, embeddings, vector store, retriever, prompt, generator.
- Build a local Chroma vector database from a Hugging Face dataset.
- Compose LangChain Expression Language (LCEL) chains with chat history.
- Diagnose retrieval quality and adjust k/chunking.
- Ship an interactive Gradio app with citations.

What you'll build

- A reproducible RAG pipeline over geoscience content.
- Conversational RAG with memory (RunnableWithMessageHistory).
- A Gradio UI to ask domain questions and see cited sources.

```
import warnings
warnings.filterwarnings('ignore')

# Environment setup
!pip -q install langchain langchain-core langchain-community langchain-huggingface
!pip -q install requests bitsandbytes transformers datasets accelerate
!pip -q install langchain-chroma python-dotenv huggingface_hub
!pip install -q --upgrade opentelemetry-api opentelemetry-sdk
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.5/2.5 MB 41.3 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.0/1.0 MB 25.9 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 64.7/64.7 kB 3.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 51.0/51.0 kB 4.5 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is
google-colab 1.0.0 requires requests==2.32.4, but you have requests 2.32.5 which is incompatible.
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.7/60.7 MB 13.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 52.0/52.0 kB 4.5 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 21.6/21.6 MB 35.9 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 278.2/278.2 kB 8.4 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.0/2.0 MB 38.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 17.2/17.2 MB 35.4 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 72.1/72.1 kB 8.1 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 142.0/142.0 kB 7.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 68.7/68.7 kB 4.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 231.6/231.6 kB 9.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 71.6/71.6 kB 3.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.6/60.6 kB 2.5 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is
opentelemetry-exporter-otlp-proto-http 1.38.0 requires opentelemetry-exporter-otlp-proto-common==1.38.0, but you have opente
opentelemetry-exporter-otlp-proto-http 1.38.0 requires opentelemetry-proto==1.38.0, but you have opentelemetry-proto 1.40.0
opentelemetry-exporter-otlp-proto-http 1.38.0 requires opentelemetry-sdk~=1.38.0, but you have opentelemetry-sdk 1.40.0 whic
opentelemetry-exporter-gcp-logging 1.11.0a0 requires opentelemetry-sdk<1.39.0,>=1.35.0, but you have opentelemetry-sdk 1.40.
google-adk 1.27.1 requires opentelemetry-api<1.39.0,>=1.36.0, but you have opentelemetry-api 1.40.0 which is incompatible.
google-adk 1.27.1 requires opentelemetry-sdk<1.39.0,>=1.36.0, but you have opentelemetry-sdk 1.40.0 which is incompatible.
```

```
# Install the classic package once in the environment:
!pip install -U langchain-classic
```

```
Requirement already satisfied: langchain-classic in /usr/local/lib/python3.12/dist-packages (1.0.3)
Requirement already satisfied: langchain-core<2.0.0,>=1.2.19 in /usr/local/lib/python3.12/dist-packages (from langchain-clas
Requirement already satisfied: langchain-text-splitters<2.0.0,>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from langc
Requirement already satisfied: langsmith<1.0.0,>=0.1.17 in /usr/local/lib/python3.12/dist-packages (from langchain-classic)
Requirement already satisfied: pydantic<3.0.0,>=2.7.4 in /usr/local/lib/python3.12/dist-packages (from langchain-classic) (2
Requirement already satisfied: pyyaml<7.0.0,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from langchain-classic) (6.0
Requirement already satisfied: requests<3.0.0,>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from langchain-classic) (2
Requirement already satisfied: sqlalchemy<3.0.0,>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from langchain-classic)
Requirement already satisfied: jsonpatch<2.0.0,>=1.33.0 in /usr/local/lib/python3.12/dist-packages (from langchain-core<2.0.
Requirement already satisfied: packaging>=23.2.0 in /usr/local/lib/python3.12/dist-packages (from langchain-core<2.0.0,>=1.2
Requirement already satisfied: tenacity!=8.4.0,<10.0.0,>=8.1.0 in /usr/local/lib/python3.12/dist-packages (from langchain-co
Requirement already satisfied: typing-extensions<5.0.0,>=4.7.0 in /usr/local/lib/python3.12/dist-packages (from langchain-co
Requirement already satisfied: uuid-utils<1.0,>=0.12.0 in /usr/local/lib/python3.12/dist-packages (from langchain-core<2.0.0
```

```
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>=0.1.17->l
Requirement already satisfied: orjson>=3.9.14 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>=0.1.17->lar
Requirement already satisfied: requests-toolbelt>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>=0
Requirement already satisfied: xxhash>=3.0.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>=0.1.17->lang
Requirement already satisfied: zstandard>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>=0.1.17->
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.7.
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.7.4
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>=2.
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.0.0->langcha
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.0.0->l
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.0.0->l
Requirement already satisfied: greenlet>=1 in /usr/local/lib/python3.12/dist-packages (from sqlalchemy<3.0.0,>=1.4.0->langch
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->langsmith<1.0.0,>=0.
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->langsmith<1.
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->l
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.12/dist-packages (from jsonpatch<2.0.0,>=1.33.0->l
```

```
# Hugging Face API token
# Retrieving the token is required to get access to HF hub
from google.colab import userdata
HF_TOKEN = userdata.get('HF_TOKEN')
```

1. What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that enhances the capabilities of a Large Language Model (LLM) by providing it with external, up-to-date, and domain-specific information. Instead of relying solely on the knowledge baked into its weights during training, the LLM can access a knowledge base to ground its answers in facts.

The core RAG workflow consists of two main phases:

1. **Indexing (Offline)**: We process our knowledge base (e.g., technical manuals, reports) by splitting documents into smaller chunks, converting them into numerical vectors (embeddings), and storing them in a specialized vector database.
2. **Retrieval & Generation (Online)**: When a user asks a question, we first retrieve the most relevant chunks from our database. Then, we feed both the question and the retrieved context to the LLM, instructing it to generate an answer based on the provided information.

Why this helps:

- **Reduces Hallucinations**: By anchoring the LLM's response to your specific data, it's less likely to invent facts.
- **Enables Domain-Specific Knowledge**: You can build a chatbot for your proprietary documents without the massive cost of fine-tuning a model.
- **Keeps Knowledge Fresh**: Updating the knowledge base is as simple as re-indexing your documents, which is much faster and cheaper than retraining an entire LLM.

```
from pathlib import Path

# Models and paths
MODEL_EMBED = "sentence-transformers/all-MiniLM-L6-v2"
LLM_NAME = "microsoft/Phi-3-mini-4k-instruct"  # local HF model
HF_DATASET = "GainEnergy/ogdataset"

WORKDIR = Path.cwd()
DATA_DIR = WORKDIR / "raw_data"
DB_DIR = WORKDIR / "local_data" / "geo_vector_db"
DB_DIR.parent.mkdir(parents=True, exist_ok=True)

CHUNK_SIZE = 1000
CHUNK_OVERLAP = 200
TOP_K = 8
```

2. Ingest and Structure the Corpus

The first step in any RAG pipeline is to prepare the knowledge base. Here, we'll download a curated geoscience dataset from the Hugging Face Hub. Each record in the dataset contains text content and associated metadata (like title and topic).

We will then transform these records into LangChain `Document` objects. This is a standard format that LangChain uses to represent pieces of text, making it easy to integrate with other components like text splitters and vector stores.

> Key Parameters (`snapshot_download`)
>
> - `repo_id`: The identifier of the Hugging Face repository (e.g., `"GainEnergy/ogdataset"`).

- `repo_type`: Specifies whether the repository is a `"dataset"` or `"model"`.
- `local_dir`: The local path where the repository files will be downloaded.
- `local_dir_use_symlinks`: Set to `False` to download the actual files instead of creating symlinks, which is more robust in environments like Google Colab.

```
# %%
from huggingface_hub import snapshot_download, login

if HF_TOKEN:
    login(token=HF_TOKEN)

DATA_DIR.mkdir(parents=True, exist_ok=True)

repo_local_dir = DATA_DIR  # place under raw_data/
snapshot_download(
    repo_id=HF_DATASET,
    repo_type="dataset",
    local_dir=str(repo_local_dir),
    local_dir_use_symlinks=False,
)

# Find training_data.json (search recursively to be robust)
json_candidates = list(repo_local_dir.rglob("training_data.json"))
if not json_candidates:
    raise FileNotFoundError("Could not find training_data.json in raw_data/ after snapshot_download.")
json_path = json_candidates[0]
print(f"Using dataset file: {json_path}")
```

```
Download complete: 53.6k/? [00:01<00:00, 29.9kB/s]

Fetching 14 files: 100% 14/14 [00:00<00:00, 29.15it/s]
Using dataset file: /content/raw_data/training_data.json
```

2.1. Load and Transform Data

Now that we have the data file, we'll load the JSON content and transform each record into a LangChain `Document`. A `Document` is a simple object that holds the text (`page_content`) and any associated `metadata` (like the source title or topic). This standardized format is crucial for compatibility with the rest of the LangChain ecosystem.

```
import json
from typing import List, Dict, Any
from langchain_core.documents import Document

with open(json_path, "r") as f:
    data: List[Dict[str, Any]] = json.load(f)

print(f"Loaded {len(data)} records from {json_path.name}")

docs: List[Document] = []
for rec in data:
    text = (rec.get("content") or "").strip()
    if not text:
        continue
    meta = {
        "id": rec.get("id"),
        "topic": rec.get("topic"),
        "title": rec.get("title"),
    }
    docs.append(Document(page_content=text, metadata=meta))

print(f"Built {len(docs)} LangChain Documents. Example metadata:", docs[0].metadata if docs else "N/A")
```

```
Loaded 120 records from training_data.json
Built 120 LangChain Documents. Example metadata: {'id': 'eng_doc_001', 'topic': 'Drilling Fluids', 'title': 'Drilling Fluids
```

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma

splitter = RecursiveCharacterTextSplitter(chunk_size=CHUNK_SIZE, chunk_overlap=CHUNK_OVERLAP)
chunks = splitter.split_documents(docs)
print(f"Split {len(docs)} documents into {len(chunks)} chunks.")

embeddings = HuggingFaceEmbeddings(model_name=MODEL_EMBED)
```

```
# Recreate vectorstore fresh (persisted locally)
if DB_DIR.exists():
    import shutil
    shutil.rmtree(DB_DIR)

vectorstore = Chroma.from_documents(documents=chunks, embedding=embeddings, persist_directory=str(DB_DIR))
print(f"Vectorstore created and persisted at: {DB_DIR}")
```

```
Split 120 documents into 120 chunks.
modules.json: 100%  349/349 [00:00<00:00, 40.7kB/s]
config_sentence_transformers.json: 100%  116/116 [00:00<00:00, 11.4kB/s]
README.md:  10.5k/? [00:00<00:00, 845kB/s]
sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 6.27kB/s]
config.json: 100%  612/612 [00:00<00:00, 67.1kB/s]
model.safetensors: 100%  90.9M/90.9M [00:01<00:00, 453MB/s]
Loading weights: 100%  103/103 [00:00<00:00, 432.90it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100%  350/350 [00:00<00:00, 28.0kB/s]
vocab.txt:  232k/? [00:00<00:00, 10.9MB/s]
tokenizer.json:  466k/? [00:00<00:00, 20.9MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 12.2kB/s]
config.json: 100%  190/190 [00:00<00:00, 19.9kB/s]
Vectorstore created and persisted at: /content/local_data/geo_vector_db
```

3. Building the RAG Pipeline

Now we assemble the core components of our RAG pipeline: the text splitter, the embedding model, and the vector store.

3.1. Text Splitting and Embedding

LLMs have a limited context window, so we can't feed them entire documents at once. We must split our documents into smaller, manageable chunks. The `RecursiveCharacterTextSplitter` is a smart way to do this, as it tries to keep related pieces of text together by splitting on paragraphs, sentences, and then characters as a last resort.

> Key Parameters (`RecursiveCharacterTextSplitter`)
>
> - `chunk_size`: The maximum number of characters in each chunk. A good starting point is 500-1000.
> - `chunk_overlap`: The number of characters to overlap between adjacent chunks. This helps maintain context across chunk boundaries. A common value is 10-20% of the chunk size.

Once split, each chunk is converted into a numerical vector using an **embedding model**. We use `sentence-transformers/all-MiniLM-L6-v2`, a fast and effective model for this task. These vectors capture the semantic meaning of the text.

3.2. Vector Store

The final step is to store these embeddings in a **vector store** for efficient retrieval. We use `Chroma`, a popular open-source vector database that can run locally. By persisting the database to disk, we can reuse it in future runs without re-indexing the documents every time.

The next cell initializes the text splitter, creates embeddings for the document chunks, and builds a persistent Chroma vector store. We include a step to remove any existing database to ensure we start fresh.

3.3. Generation Pipeline

With our knowledge base indexed, we now need the "G" in RAG: the **Generator**. This is the LLM that will synthesize an answer based on the user's question and the retrieved context. We'll load `microsoft/Phi-3-mini-4k-instruct`, a powerful yet relatively small model suitable for running in Colab.

To optimize performance, we'll use 4-bit quantization via `bitsandbytes`, which significantly reduces the model's memory footprint with minimal impact on quality. The `build_generation_pipeline` function below handles device detection (CUDA for NVIDIA GPUs, MPS for Apple Silicon, CPU as a fallback) and configures the model and tokenizer accordingly.

Finally, we wrap the Hugging Face `pipeline` in a `LangChainHuggingFace` object to make it compatible with the LangChain Expression Language (LCEL).

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline

def build_generation_pipeline(model_id: str):
    use_cuda = torch.cuda.is_available()
    use_mps = getattr(torch.backends, "mps", None) and torch.backends.mps.is_available()

    tokenizer = AutoTokenizer.from_pretrained(model_id)

    quantization_config = None
    model_kwargs = {}
    if use_cuda:
        # Optional 4-bit if bitsandbytes available
        try:
            from transformers import BitsAndBytesConfig
            quantization_config = BitsAndBytesConfig(load_in_4bit=True)
            model_kwargs["quantization_config"] = quantization_config
            model_kwargs["device_map"] = "auto"
            print("Using CUDA with 4-bit quantization.")
        except Exception:
            model_kwargs["device_map"] = "auto"
            model_kwargs["torch_dtype"] = torch.float16
            print("Using CUDA without bitsandbytes 4-bit (fallback).")
    elif use_mps:
        model_kwargs["torch_dtype"] = torch.float16
        model_kwargs["device_map"] = "auto"
        print("Using Apple MPS (float16).")
    else:
        model_kwargs["torch_dtype"] = torch.float32
        model_kwargs["device_map"] = "auto"
        print("Using CPU (this will be slow).")

    model = AutoModelForCausalLM.from_pretrained(model_id, **model_kwargs)

    # Ensure pad token is set
    if tokenizer.pad_token_id is None:
        tokenizer.pad_token_id = tokenizer.eos_token_id

    gen_pipe = pipeline(
        "text-generation",
        model=model,
        tokenizer=tokenizer,
        max_new_tokens=256,
        temperature=0.2,
        do_sample=False,
        pad_token_id=tokenizer.pad_token_id,
        return_full_text=False,
    )
    return gen_pipe

gen_pipe = build_generation_pipeline(LLM_NAME)

# Wrap as LangChain LLM
from langchain_huggingface import HuggingFacePipeline
llm = HuggingFacePipeline(pipeline=gen_pipe)
```

```
config.json: 100%                                                    967/967 [00:00<00:00, 93.2kB/s]
tokenizer_config.json:        3.44k/? [00:00<00:00, 288kB/s]
tokenizer.json:        1.94M/? [00:00<00:00, 57.8MB/s]
tokenizer.model: 100%                                                500k/500k [00:00<00:00, 2.19MB/s]
added_tokens.json: 100%                                              306/306 [00:00<00:00, 27.9kB/s]
special_tokens_map.json: 100%                                        599/599 [00:00<00:00, 71.8kB/s]
Using CUDA with 4-bit quantization.
model.safetensors.index.json:        16.5k/? [00:00<00:00, 1.20MB/s]
Download complete: 100%       7.64G/7.64G [02:27<00:00, 65.2MB/s]
Fetching 2 files: 100%                                               2/2 [02:27<00:00, 147.29s/it]
Loading weights: 100%                                                195/195 [00:23<00:00, 72.94it/s, Materializing param=model.norm.weight]
generation_config.json: 100%                                         181/181 [00:00<00:00, 22.2kB/s]
The following generation flags are not valid and may be ignored: ['temperature']. Set `TRANSFORMERS_VERBOSITY=info` for more
Passing `generation_config` together with generation-related arguments=({'temperature', 'max_new_tokens', 'pad_token_id', 'd
```

3.4. Composing the RAG Chain with Memory

Now, we'll tie everything together using the **LangChain Expression Language (LCEL)**. This declarative style makes the flow of data transparent and easy to modify. Here are the key steps:

1. **Retriever**: We create a retriever from our `vectorstore`. This component takes a user query, embeds it, and retrieves the most relevant document chunks from the vector database. The `search_kwargs={"k": TOP_K}` parameter controls how many chunks to retrieve.
2. **Prompt Template**: We design a `ChatPromptTemplate` to structure the input for the LLM. It includes:
   - A `system` message to define the assistant's persona and instructions.
   - A `MessagesPlaceholder` to inject the conversation history.
   - A `human` message that combines the retrieved `{context}` and the user's `{input}`.
3. **LCEL Chain**: We compose our chain using the `|` (pipe) operator. This is a more modern and explicit way to define the RAG flow compared to using helper functions like `create_retrieval_chain`.
   - We use `RunnablePassthrough.assign` to pass the retrieved documents into the `context` key for the prompt.
   - The `RunnableParallel` dictionary (`{"context": retriever, "input": RunnablePassthrough()}`) is a key step. It invokes the retriever and passes the original user input through simultaneously.
4. **Memory**: To enable multi-turn conversations, we wrap our RAG chain with `RunnableWithMessageHistory`. This powerful runnable automatically manages chat history. It uses a `get_history` function to load and save messages for a given `session_id`, allowing the bot to remember previous turns in the conversation.

```
# %%
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
#from langchain.chains.combine_documents import create_stuff_documents_chain
#from langchain.chains import create_retrieval_chain
from langchain_classic.chains.combine_documents import create_stuff_documents_chain
from langchain_classic.chains import create_retrieval_chain
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_community.chat_message_histories import ChatMessageHistory

retriever = vectorstore.as_retriever(search_kwargs={"k": TOP_K})

prompt = ChatPromptTemplate.from_messages(
    [
        ("system", "You are a helpful assistant for geology and petroleum engineering. "
                   "Use the provided context to answer the question. If unsure, say you don't know. "
                   "Cite titles when possible."),
        MessagesPlaceholder("chat_history"),
        ("human", "Context:\n{context}\n\nQuestion: {input}"),
    ]
)

# Stuff documents into the prompt
document_chain = create_stuff_documents_chain(llm, prompt)
rag_chain = create_retrieval_chain(retriever, document_chain)  # returns {"answer", "context"}

# Attach message history
_store: dict[str, ChatMessageHistory] = {}
```

```
def get_history(session_id: str) -> ChatMessageHistory:
    if session_id not in _store:
        _store[session_id] = ChatMessageHistory()
    return _store[session_id]

conv_rag = RunnableWithMessageHistory(
    rag_chain,
    get_history,
    input_messages_key="input",
    history_messages_key="chat_history",
    output_messages_key="answer",
)

print("RAG chain with chat history initialized.")
```

```
RAG chain with chat history initialized.
```

4. Querying and Diagnostics

With the conversational RAG chain built, we can now ask it questions. We'll also create some diagnostic functions to inspect the retrieval process, which is crucial for tuning and troubleshooting.

```
from typing import Tuple, List

def ask(question: str, session_id: str = "default") -> Tuple[str, List[Document]]:
    result = conv_rag.invoke(
        {"input": question},
        config={"configurable": {"session_id": session_id}},
    )
    answer = result.get("answer", "")
    source_docs = result.get("context", []) # list[Document]
    print("Answer:\n", answer)
    print("\nCitations:")
    for i, d in enumerate(source_docs, 1):
        md = d.metadata or {}
        title = md.get("title") or md.get("topic") or "Untitled"
        print(f"[{i}] {title}")
    return answer, source_docs

def rag_query(query: str, k: int = 3) -> str:
    # Build a temporary retrieval chain to allow custom k
    tmp_retriever = vectorstore.as_retriever(search_kwargs={"k": k})
    tmp_chain = create_retrieval_chain(tmp_retriever, document_chain)
    result = tmp_chain.invoke({"input": query})
    if isinstance(result, dict):
        return result.get("answer", "")
    return str(result)
```

```
# 3. Retrieval diagnostics: see what you're feeding the LLM
from textwrap import shorten

def preview_retrieval(query: str, k: int = 5):
    print(f"Query: {query}\n---")
    docs_scores = []
    try:
        # Chroma supports similarity_search_with_score
        docs_scores = vectorstore.similarity_search_with_score(query, k=k)
    except Exception:
        # Fallback (no scores)
        docs = vectorstore.similarity_search(query, k=k)
        docs_scores = [(d, None) for d in docs]

    for i, (doc, score) in enumerate(docs_scores, 1):
        md = doc.metadata or {}
        title = md.get("title") or md.get("topic") or "Untitled"
        snippet = shorten(doc.page_content, width=180, placeholder=" ...")
        score_str = f" | score={score:.4f}" if score is not None else ""
        print(f"[{i}] {title}{score_str}\n    {snippet}\n")

# Example diagnostics
preview_retrieval("What factors control porosity and permeability in clastic reservoirs?", k=5)
```

```
Query: What factors control porosity and permeability in clastic reservoirs?
---
[1] Reservoir Simulation Guide | score=0.8572
    Reservoir simulation models fluid flow behavior in porous media. Input data includes rock properties, fluid properties,

[2] Formation Evaluation Manual | score=0.9500
```

```
    Formation evaluation involves analyzing well logs, cores, and drilling data to determine reservoir properties. Key param

[3] Well Testing Procedures | score=1.1315
    Well testing provides reservoir data including pressure, permeability, and boundaries. Tests include buildup, drawdown,

[4] Completion Design Guide | score=1.1756
    Well completion design includes tubing selection, packer placement, and downhole equipment configuration. Considers rese

[5] Buoyancy Factor | score=1.1927
    Buoyancy Factor = (65.5 - Mud Weight)/65.5. Used to calculate effective weight of tubulars in drilling fluid.
```

The `preview_retrieval` function is a vital diagnostic tool. It lets you see exactly which document chunks are being retrieved for a given query and what their similarity scores are. This helps you answer critical questions:

- Are the retrieved chunks relevant to the query?
- Is `k` (the number of chunks) too high or too low?
- Is the `chunk_size` appropriate? If snippets are too short or too long, you may need to adjust your splitting strategy.

```
# %%
test_questions = [
    "What factors control porosity and permeability in clastic reservoirs?",
    "What standards apply to BOP?",
]
for q in test_questions:
    print("\n=== Q:", q)
    ask(q)
```

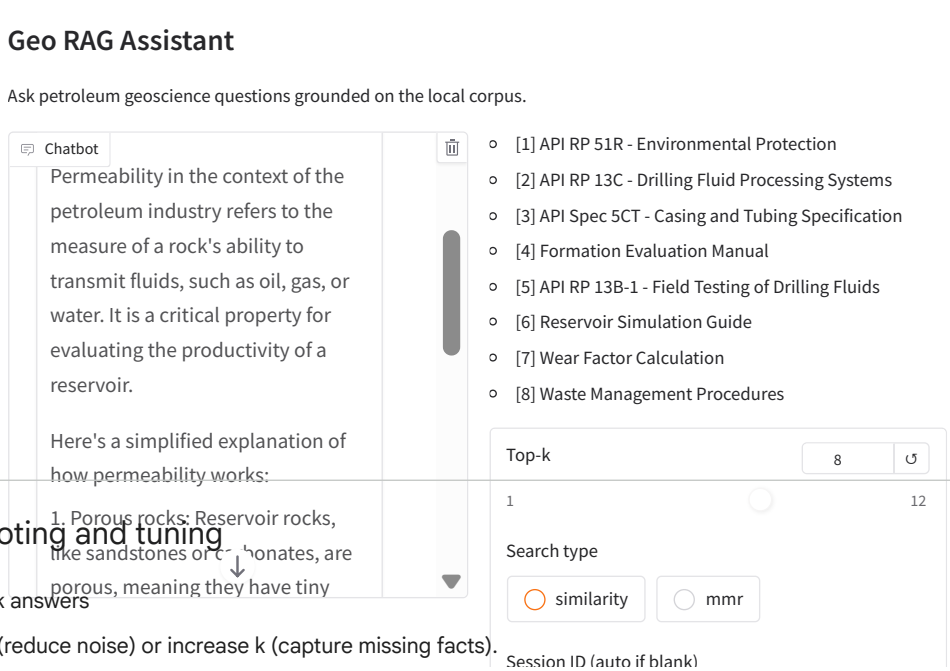
```
Factors Controlling Porosity

Grain Size

The size of the clastic particles significantly influences porosity. Coarser-grained rocks, such

Citations:
[1] Reservoir Simulation Guide
[2] Formation Evaluation Manual
[3] Well Testing Procedures
[4] Completion Design Guide
[5] Buoyancy Factor
[6] Enhanced Oil Recovery Guide
[7] Hydrostatic Pressure Calculation
[8] Casing Design Manual

=== Q: What standards apply to BOP?
Answer:


Answer:

The API Standard 53 applies to blowout preventers (BOPs). It outlines the requirements for BOPs used in drilling and well


Document:

Document:

API Standard 53: Blowout Preventer (BOP) Requirements

Abstract
```

4.2. Testing the Full Chain

Let's run our test questions through the complete `conv_rag` chain to see the final, context-aware answers and their cited sources.

```
# %%
# 5. Gradio app: Conversational RAG with citations
import uuid
import gradio as gr
#from langchain.chains import create_retrieval_chain
from langchain_classic.chains import create_retrieval_chain

def build_conv(k: int = TOP_K, search_type: str = "similarity"):
    # retr = vectorstore.as_retriever(search_kwargs={"k": k, "search_type": search_type}) # Chroma does not support search_t
    retr = vectorstore.as_retriever(search_kwargs={"k": k})
    chain = create_retrieval_chain(retr, document_chain)
    conv = RunnableWithMessageHistory(
        chain,
        get_history,
        input_messages_key="input",
        history_messages_key="chat_history",
        output_messages_key="answer",
    )
    return conv

def format_citations(docs: List[Document]) -> str:
    lines = []
    seen = set()
    for i, d in enumerate(docs, 1):
        md = d.metadata or {}
        title = md.get("title") or md.get("topic") or "Untitled"
        if title in seen:
            continue
        seen.add(title)
        lines.append(f"- [{i}] {title}")
    return "\n".join(lines) if lines else "_No citations_"

def respond(message, chat_history, sid, k, search_type):
    if not sid:
        sid = str(uuid.uuid4())
    conv = build_conv(k=int(k), search_type=search_type)
    result = conv.invoke({"input": message}, config={"configurable": {"session_id": sid}})
    answer = result.get("answer", "")
    ctx = result.get("context", [])
    citations_md = format_citations(ctx)
    chat_history = chat_history + [[message, answer]]
    return chat_history, sid, citations_md

with gr.Blocks(title="Geo RAG Assistant") as demo:
    gr.Markdown("## Geo RAG Assistant\nAsk petroleum geoscience questions grounded on the local corpus.")
    with gr.Row():
        chatbot = gr.Chatbot(height=350)
        with gr.Column():
            citations = gr.Markdown(value="_Citations will appear here_")
            k_slider = gr.Slider(1, 12, value=TOP_K, step=1, label="Top-k")
            search_type = gr.Radio(choices=["similarity", "mmr"], value="similarity", label="Search type")
            sid = gr.Textbox(value="", label="Session ID (auto if blank)")

    msg = gr.Textbox(placeholder="Type your question about porosity, BOP standards, etc.")
    send = gr.Button("Ask")
    clear = gr.Button("Clear chat")

    def on_clear():
        return [], "", "_Citations will appear here_"

    send.click(respond, [msg, chatbot, sid, k_slider, search_type], [chatbot, sid, citations])
    clear.click(on_clear, outputs=[chatbot, sid, citations])

try:
    demo.launch(share=False)
except Exception as e:
    print("Gradio failed to launch in this environment:", e)
```

```
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
Note: opening Chrome Inspector may crash demo inside Colab notebooks.
* To create a public link, set `share=True` in `launch()`.
Running on https://localhost:7860/
```



5. Troubleshooting and tuning

- Empty or weak answers
  - Lower k (reduce noise) or increase k (capture missing facts).
  - Reduce CHUNK_SIZE or increase CHUNK_OVERLAP for finer granularity.
  - Verify dataset loaded: print head of docs and metadata.
- Slow generation
  - Lower max_new_tokens in the HF pipeline.
  - Use smaller models; ensure MPS/CUDA is active.
  - Reduce k to fetch fewer chunks.
- macOS (MPS) quantization
  - BitsAndBytes 4-bit is not supported; use float16 on MPS or CPU float32.
  - Switch to small models (≤2B) if resources are limited.
- Re-running faster
  - Skip re-embedding by reusing the persisted Chroma DB (don't delete DB_DIR).
  - Cache dataset snapshot on disk.
- Better citations